

`number.php` is shown below. At this point, also remember the naming convention of HTML files. If the HTML file contains embedded PHP script, then the extension of the HTML file is `.php`, and if there is no embedded PHP script inside an HTML script, then the extension of the file is `.html`.

```
<html>
<body>
<?php
    $pat1 = '/^[+-]?[d+$/]';
    $pat2 = '/^[+-]?[d*\\.d+$/]';
    $str = $_POST["number"];
    if(preg_match($pat1,$str))
    {
        echo '<b> Integer </b>';
    }
    elseif(preg_match($pat2,$str))
    {
        echo '<b> Real Number </b>';
    }
    else
    {
        echo '<b> Not a number </b>';
    }
    ?>
</body>
</html>
```

The HTML section of the file only contains the tags `<html>` and `<body>` and their meaning is obvious. But the PHP script in the file requires some explaining. There are two regular expression patterns defined by the PHP script stored in the variables `$pat1` and `$pat2`. If you examine the two regular expression patterns carefully, you will understand the benefits of using `preg` which is based on PCRE. I have reused the same regular expression patterns we have discussed in the earlier article dealing with Perl. The line of code `'$pat1 = '/^[+-]?[d+$/]';` defines a regular expression pattern that matches any integer. Even integers like +111, -222, etc, will be matched by this regular expression.

The next line of code `'$pat2 = '/^[+-]?[d*\\.d+$/]';` defines a regular expression pattern that matches real numbers. Here again, we are only identifying a subset of real numbers called rational numbers. But then again, let us not be too mathematical. For a detailed discussion of these regular expressions, refer to the earlier article on Perl, in this series. The best part is that any regular expression that we have developed there can be used in PHP without making changes. I have made a slight change in the second regular expression pattern `/^[+-]?[d*\\.d+$/]` to accommodate real numbers of the form .333 also. The original Perl regular expression was `/^[+-]?[d*\\.d+$/]` which will only validate real numbers like 0.333 and not .333.

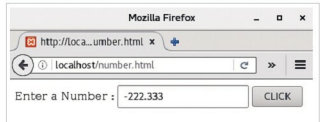


Figure 3: HTML page from `number.html`

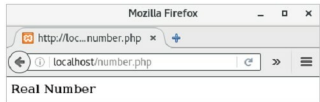


Figure 4: Output of `number.php`

The next line of code `'$str = $_POST["number"];` reads the input data from the HTML file `number.html` and stores it in the variable `$str`. The next few lines of code contain an *if-else* block which matches the input text with the two regular expression patterns. The function `preg_match()` is used in the *if* statement and the *elseif* statement to search for a match. Depending on the results of these matches, the PHP script prints the suitable message in bold text in the Web browser. To view the output of the HTML script, open a Web browser and on the address bar, type `localhost/number.html`. The resulting HTML page is shown in Figure 3. Enter a number in the text field and press the *Submit* button. You will see one of the three possible output messages on the Web page — ‘Integer’, ‘Real Number’, or ‘Not a number’. Figure 4 shows the output obtained when the number -222.333 is given as input.

Now that we have discussed a useful regular expression, it is time to wind up the article. Here, I have discussed the programming language PHP almost as much as the regular expressions in it. I believe the whole point of this series is to explore how regular expressions work in different programming languages by analysing the features of those programming languages rather than discussing regular expressions in a language-agnostic way. And now that we have covered PHP regular expressions, I am sure you will have some idea about using regular expressions on the server side. But what about regular expressions on the client side? In the last example, the validation could have been done on the client side itself rather than sending the data all the way to the server. So, in the next article in this series, we will discuss the use of regular expressions in JavaScript – a client-side scripting language. 🐼

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. He maintains a technical blog at www.computingforbeginners.blogspot.in and can be reached at deepump@hotmail.com.